Bellman-Ford, Warshall's and Floyd-Warshall Algorithms

Negative Weights, Transitive Closure, All-Pairs Shortest Paths and C Implementations

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

Recap: Graph Algorithms

Previously, we studied:

- Minimum spanning trees.
- Kruskal's algorithm.
- Prim's algorithm.
- Single-source shortest paths.
- Dijkstra's algorithm.
- Edge relaxation.
- Distance and parent arrays.

Today's Focus

What happens when edge weights are negative?
How do we compute reachability and shortest paths for all pairs?

Types of Shortest-Path Problems

1. **Single-source shortest paths**
   Shortest paths from one source to every reachable vertex.
2. **Single-pair shortest path**
   Shortest path between a specified source and destination.
3. **All-pairs shortest paths**
   Shortest paths between every pair of vertices.

Shortest-Path Algorithm Overview

Algorithm	Main Purpose
BFS	Shortest paths in unweighted graphs
Dijkstra	Single-source, nonnegative weights
Bellman-Ford	Single-source, negative weights allowed
Warshall	Reachability / transitive closure
Floyd-Warshall	All-pairs shortest-path distances

Understanding Negative Edge Weights

A negative edge represents a negative contribution to the total path cost.
For example:

$$w(A, B) = 5$$

$$w(B, C) = -3$$

Then:

$$Cost(A \to B \to C) = 5 + (-3)$$

$$\boxed{Cost = 2}$$

Important

Negative edge weights are meaningful in directed graphs, for example when representing credits or adjustments.

Why Dijkstra May Fail

Consider:

$$A \to B = 2$$

$$A \to C = 5$$

$$C \to B = -6$$

The direct path gives:

$$dist[B] = 2$$

But the path through C gives:

$$5 - 6 = -1$$

Problem

A standard Dijkstra implementation may finalize B before discovering the better path through C.

What Is a Negative-Weight Cycle?

Definition

A negative-weight cycle is a directed cycle whose total edge weight is negative.

Example:

$$A \to B = 2$$

$$B \to C = -5$$

$$C \to A = 1$$

Total cycle weight:

$$2 - 5 + 1 = \boxed{-2}$$

Why Are Negative Cycles Important?

If a negative cycle is reachable from the source, traversing it repeatedly reduces the path cost without bound.

$$-2,\ -4,\ -6,\ -8, \ldots$$

Consequence

No finite shortest-path distance exists for vertices reachable from such a cycle.

Bellman-Ford can detect reachable negative-weight cycles.

Introduction to Bellman-Ford

Definition

Bellman-Ford computes shortest-path distances from a single source by repeatedly relaxing every edge.

- Works with negative edge weights.
- Supports directed graphs.
- Detects reachable negative cycles.
- Uses repeated relaxation.

Bellman-Ford: Main Idea

A shortest simple path in a graph with V vertices contains at most:

$$\boxed{V - 1 \text{ edges}}$$

Therefore, Bellman-Ford performs up to:

$$\boxed{V - 1 \text{ relaxation passes}}$$

A further pass checks for reachable negative cycles.

What Is Relaxation?

For a directed edge:

$$u \to v$$

with weight $w(u, v)$, update if:

$$dist[u] + w(u, v) < dist[v]$$

Then:

$$\boxed{dist[v] = dist[u] + w(u, v)}$$

and:

$$parent[v] = u$$

Only relax if u is reachable.

Bellman-Ford: Algorithm Steps

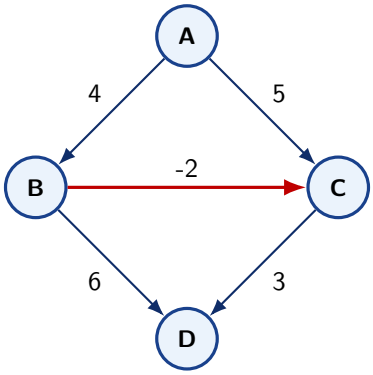
1. Set source distance to zero.
2. Set all other distances to infinity.
3. Repeat $V - 1$ times:
   - Examine every edge.
   - Relax the edge if possible.
4. Perform one additional edge scan.
5. If a distance can still decrease, report a reachable negative cycle.

Bellman-Ford Pseudocode

Algorithm: BellmanFord(G, source)

```
For each vertex v:
  dist[v] = INFINITY
  parent[v] = NIL
dist[source] = 0
Repeat V-1 times:
  For each edge (u,v,w):
    If dist[u] != INFINITY and
    dist[u] + w < dist[v]:
      dist[v] = dist[u] + w
      parent[v] = u
For each edge (u,v,w):
  If relaxation is still possible:
    Report negative cycle
```

Weighted Directed Graph



Task

Apply Bellman-Ford with source A.

Edge List and Processing Order

Directed Edge	Weight
$A \rightarrow B$	4
$A \rightarrow C$	5
$B \rightarrow C$	-2
$B \rightarrow D$	6
$C \rightarrow D$	3

Important

We process edges in the order shown above during each relaxation pass.

Initialization

Initially:

$$dist[A] = 0$$

$$dist[B] = \infty$$

$$dist[C] = \infty$$

$$dist[D] = \infty$$

Vertex	A	B	C	D
Distance	0	∞	∞	∞

First Pass: Relax A to B

Edge:

$$A \to B = 4$$

Candidate:

$$0 + 4 = 4$$

Since:

$$4 < \infty$$

update:

$$\boxed{dist[B] = 4}$$

First Pass: Relax A to C

Edge:

$$A \to C = 5$$

Candidate:

$$0 + 5 = 5$$

Update:

$$\boxed{dist[C] = 5}$$

First Pass: Relax B to C

Edge:

$$B \to C = -2$$

Candidate:

$$4 + (-2) = 2$$

Since:

$$2 < 5$$

update:

$$\boxed{dist[C] = 2}$$

First Pass: Relax B to D

Edge:

$$B \to D = 6$$

Candidate:

$$4 + 6 = 10$$

Update:

$$\boxed{dist[D] = 10}$$

First Pass: Relax C to D

Edge:

$$C \to D = 3$$

Candidate:

$$2 + 3 = 5$$

Since:

$$5 < 10$$

update:

$$\boxed{dist[D] = 5}$$

Bellman-Ford: Pass Results

Pass	A	B	C	D
Initial	0	∞	∞	∞
1	0	4	2	5
2	0	4	2	5
3	0	4	2	5

Observation

No distance improves after the first pass for this edge-processing order.

Final Bellman-Ford Result

Shortest Distances from A

$$\boxed{A = 0, \quad B = 4, \quad C = 2, \quad D = 5}$$

Shortest path to D:

$$A \to B \to C \to D$$

Total:

$$4 + (-2) + 3 = \boxed{5}$$

Bellman-Ford C Implementation

The program uses:

- An edge-list representation.
- Distance array.
- Parent array.
- Repeated relaxation.
- Negative-cycle detection.

Advantages

An edge list is convenient because every pass processes all edges directly.

Bellman-Ford in C: Part 1

```c
#include <stdio.h>
#include <limits.h>

#define V 4
#define E 5
#define INF (LLONG_MAX / 4)

typedef struct {
    int u, v, weight;
} Edge;

Edge edges[E] = {
    {0, 1, 4},
    {0, 2, 5},
    {1, 2, -2},
    {1, 3, 6},
    {2, 3, 3}
};
```

Bellman-Ford in C: Part 2

```c
void bellmanFord(int source) {
    long long dist[V];
    int parent[V];

    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        parent[i] = -1;
    }

    dist[source] = 0;

    for (int pass = 1; pass < V; pass++) {
        int changed = 0;

        for (int j = 0; j < E; j++) {
            int u = edges[j].u;
            int v = edges[j].v;
            int w = edges[j].weight;

            if (dist[u] != INF &&
                dist[u] + w < dist[v]) {

                dist[v] = dist[u] + w;
                parent[v] = u;
                changed = 1;
            }
        }

        if (!changed)
            break;
    }
```

Bellman-Ford in C: Part 3

```
    for (int j = 0; j < E; j++) {
        int u = edges[j].u;
        int v = edges[j].v;
        int w = edges[j].weight;

        if (dist[u] != INF &&
            dist[u] + w < dist[v]) {

            printf("Reachable negative cycle found.\n");
            return;
        }
    }

    printf("Vertex Distance Parent\n");

    for (int i = 0; i < V; i++) {
        if (dist[i] == INF) {
            printf("%c INF -\n",
                   'A' + i);
        }
        else if (parent[i] == -1) {
            printf("%c %lld -\n",
                   'A' + i, dist[i]);
        }
        else {
            printf("%c %lld %c\n",
                   'A' + i, dist[i],
                   'A' + parent[i]);
        }
    }
}

int main(void) {
    bellmanFord(0);
    return 0;
}
```

Bellman-Ford: Expected Output

Program Output

```
Vertex Distance Parent
A 0 -
B 4 A
C 2 B
D 5 C
```

Negative-Cycle Detection

After completing the relaxation passes, scan every edge again.
If:

$$dist[u] + w(u, v) < dist[v]$$

for an edge reachable from the source, then a negative cycle is reachable.

Conclusion

A reachable negative-weight cycle prevents finite shortest distances for vertices affected by that cycle.

Bellman-Ford Complexity

For each of $V - 1$ passes, the algorithm scans up to E edges.

$$\boxed{T(V, E) = O(VE)}$$

Distance and parent arrays require:

$$\boxed{O(V)}$$

auxiliary space.
The edge list requires:

$$O(E)$$

storage.

Bellman-Ford vs Dijkstra

Feature	Dijkstra	Bellman-Ford
Approach	Greedy	Repeated relaxation
Negative edges	Not supported generally	Supported
Negative-cycle detection	No	Yes, if reachable
Matrix / basic time	$O(V^2)$	$O(VE)$
Main structure	Distance + selection	Edge list

What Is Reachability?

Definition

A vertex v is reachable from u if a directed path exists from u to v.

For example:

$$A \to B \to C$$

Then:

$$A \text{ can reach } B$$

and:

$$A \text{ can reach } C$$

even if no direct edge exists from A to C.

What Is a Path Matrix?

A path matrix records whether one vertex can reach another.

$$P[i][j] = \begin{cases} 1, & \text{if a path exists from } i \text{ to } j \\ 0, & \text{otherwise} \end{cases}$$

Convention

In this presentation, each vertex is considered reachable from itself through a zero-length path. Therefore:

$$P[i][i] = 1$$

Introduction to Warshall's Algorithm

Definition

Warshall's algorithm computes the transitive closure of a directed graph.

It determines whether a path exists between every pair of vertices.

Important

Warshall's algorithm computes reachability, not weighted shortest-path distances.

Warshall's Recurrence

Let:

$$P^{(k)}[i][j]$$

indicate whether j is reachable from i using intermediate vertices from the first k vertices.
Then:

$$\boxed{P^{(k)}[i][j] = P^{(k-1)}[i][j] \vee \left(P^{(k-1)}[i][k] \wedge P^{(k-1)}[k][j]\right)}$$

where:

$$\vee = \text{OR}, \qquad \wedge = \text{AND}$$

Warshall's Main Idea

For each intermediate vertex k, consider every pair (i, j).
A path from i to j exists if:

1. A path already exists, or
2. A path exists from i to k and from k to j.

$$\boxed{P[i][j] = P[i][j] \vee (P[i][k] \wedge P[k][j])}$$

Warshall's Pseudocode

Algorithm: Warshall(P)

```
Initialize P from adjacency matrix
Set P[i][i] = 1 for every i
For k = 0 to V-1:
  For i = 0 to V-1:
    For j = 0 to V-1:
      P[i][j] = P[i][j] OR
        (P[i][k] AND P[k][j])
```

Warshall Example: Directed Graph



Question

Which vertices are reachable from each starting vertex?

Initial Reachability Matrix

For:

$$A \to B \to C \to D$$

initialize diagonal entries to 1.

$$P^{(0)} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Observation

Initially, only direct connections and zero-length self-paths are represented.

Using A as Intermediate Vertex

Consider vertex A as the permitted intermediate.
No additional paths appear.

$$P^{(1)} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Reason

No other vertex reaches A, so using A cannot create a new path between other vertices.

Using B as Intermediate Vertex

Now allow paths through B.
Since:

$$A \to B$$

and:

$$B \to C$$

we obtain:

$$A \to C$$

$$P^{(2)} = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Using C as Intermediate Vertex

Allow paths through C.
Since:

$$A \to C \to D$$

and:

$$B \to C \to D$$

we obtain:

$$A \to D$$

$$B \to D$$

$$P^{(3)} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Using D as Intermediate Vertex

No further paths appear.

$$P^{(4)} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Final Result

The matrix is the transitive closure of the graph.

Warshall in C: Complete Program

```c
#include <stdio.h>

#define V 4

int main(void) {
    int reach[V][V] = {
        {0, 1, 0, 0},
        {0, 0, 1, 0},
        {0, 0, 0, 1},
        {0, 0, 0, 0}
    };

    for (int i = 0; i < V; i++)
        reach[i][i] = 1;

    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                reach[i][j] =
                    reach[i][j] ||
                    (reach[i][k] && reach[k][j]);
            }
        }
    }

    printf("Transitive Closure:\n");

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++)
            printf("%d ", reach[i][j]);

        printf("\n");
    }

    return 0;
}
```

Warshall: Expected Output

Program Output

```
Transitive Closure:
1 1 1 1
0 1 1 1
0 0 1 1
0 0 0 1
```

Warshall's Complexity

Three nested loops are used:

$$k = 0, \ldots, V - 1$$

$$i = 0, \ldots, V - 1$$

$$j = 0, \ldots, V - 1$$

Therefore:

$$\boxed{T(V) = O(V^3)}$$

The reachability matrix occupies:

$$\boxed{O(V^2)}$$

space.

Applications of Warshall's Algorithm

1. Network reachability.
2. Dependency analysis.
3. Directed graph connectivity.
4. Database relationship queries.
5. Access-control relationship analysis.
6. State-transition systems.

Introduction to Floyd-Warshall

Definition

Floyd-Warshall computes shortest-path distances between every pair of vertices in a weighted graph.

- Uses dynamic programming.
- Works with directed graphs.
- Supports negative edge weights.
- Requires no negative cycles for finite all-pairs distances.

Warshall vs Floyd-Warshall

Feature	Warshall	Floyd-Warshall
Purpose	Reachability	Shortest distances
Matrix entries	0 or 1	Weights / infinity
Main operation	OR / AND	Minimum / Addition
Time complexity	$O(V^3)$	$O(V^3)$

Floyd-Warshall Recurrence

Let:

$$D^{(k)}[i][j]$$

be the shortest-path distance from i to j using intermediate vertices among the first k vertices. Then:

$$\boxed{D^{(k)}[i][j] = \min\left(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\right)}$$

Floyd-Warshall: Main Idea

For each intermediate vertex k, compare:

1. The current shortest distance from i to j.
2. The distance through k:

$$D[i][k] + D[k][j]$$

Choose the smaller value.

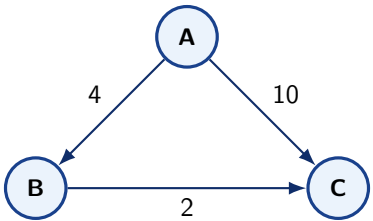
$$\boxed{D[i][j] = \min(D[i][j], D[i][k] + D[k][j])}$$

Floyd-Warshall Pseudocode

Algorithm: FloydWarshall(D)

```
Initialize distance matrix D
D[i][i] = 0
For k = 0 to V-1:
  For i = 0 to V-1:
    For j = 0 to V-1:
      If D[i][k] and D[k][j]
      are both finite:
        D[i][j] = min(
          D[i][j],
          D[i][k] + D[k][j])
After completion:
  If D[i][i] < 0 for any i:
    Negative cycle exists
```

Weighted Graph Example



Task

Find all-pairs shortest-path distances.

Initial Distance Matrix

The graph contains:

$$A \to B = 4$$

$$B \to C = 2$$

$$A \to C = 10$$

Initial matrix:

$$D^{(0)} = \begin{bmatrix} 0 & 4 & 10 \\ \infty & 0 & 2 \\ \infty & \infty & 0 \end{bmatrix}$$

Using A as Intermediate

Consider A as intermediate.
No new shorter paths are found.

$$D^{(1)} = \begin{bmatrix} 0 & 4 & 10 \\ \infty & 0 & 2 \\ \infty & \infty & 0 \end{bmatrix}$$

Observation

No other vertex has a finite path to A.

Using B as Intermediate

Consider:

$$A \to B \to C$$

Candidate distance:

$$4 + 2 = 6$$

Compare:

$$\min(10, 6) = 6$$

Therefore:

$$\boxed{D[A][C] = 6}$$

Updated matrix:

$$D^{(2)} = \begin{bmatrix} 0 & 4 & 6 \\ \infty & 0 & 2 \\ \infty & \infty & 0 \end{bmatrix}$$

Using C as Intermediate

No additional improvements occur.

$$D^{(3)} = \begin{bmatrix} 0 & 4 & 6 \\ \infty & 0 & 2 \\ \infty & \infty & 0 \end{bmatrix}$$

Final Result

$$\boxed{dist(A, C) = 6}$$

Shortest path:

$$A \to B \to C$$

Floyd-Warshall: Complete Dry Run

Stage	Distance Matrix
Initial	$\begin{bmatrix} 0 & 4 & 10 \\ \infty & 0 & 2 \\ \infty & \infty & 0 \end{bmatrix}$
Through A	$\begin{bmatrix} 0 & 4 & 10 \\ \infty & 0 & 2 \\ \infty & \infty & 0 \end{bmatrix}$
Through B	$\begin{bmatrix} 0 & 4 & 6 \\ \infty & 0 & 2 \\ \infty & \infty & 0 \end{bmatrix}$
Through C	$\begin{bmatrix} 0 & 4 & 6 \\ \infty & 0 & 2 \\ \infty & \infty & 0 \end{bmatrix}$

Floyd-Warshall in C: Complete Program

```c
#include <stdio.h>

#define V 3
#define INF 1000000000

int main(void) {
    int dist[V][V] = {
        {0, 4, 10},
        {INF, 0, 2},
        {INF, INF, 0}
    };

    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INF &&
                    dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] <
                    dist[i][j]) {

                    dist[i][j] =
                        dist[i][k] + dist[k][j];
                }
            }
        }
    }

    for (int i = 0; i < V; i++) {
        if (dist[i][i] < 0) {
            printf("Negative cycle detected.\n");
            return 0;
        }
    }

    printf("Shortest Distance Matrix:\n");
```

Floyd-Warshall: Expected Output

Program Output

```
Shortest Distance Matrix:
0 4 6
INF 0 2
INF INF 0
```

Negative Cycles in Floyd-Warshall

After running Floyd-Warshall, inspect diagonal entries:

$$D[i][i]$$

If:

$$\boxed{D[i][i] < 0}$$

for any vertex i, the graph contains a negative-weight cycle.

Important

Shortest-path distances involving a reachable negative cycle may not be finite.

Floyd-Warshall Complexity

Three nested loops:

$$k,\ i,\ j$$

each execute up to V times.
Therefore:

$$\boxed{T(V) = O(V^3)}$$

The distance matrix requires:

$$\boxed{S(V) = O(V^2)}$$

The algorithm can update the matrix in place.

Applications of Floyd-Warshall

1. All-pairs network routing.
2. Transportation cost analysis.
3. Distance matrices.
4. Small and medium weighted graphs.
5. Relationship-cost analysis.
6. Graph optimization problems.

Shortest-Path Algorithm Comparison

Algorithm	Problem	Negative Edges	Time
BFS	Single-source, unweighted	N/A	$O(V + E)$
Dijkstra	Single-source	No	$O(V^2)$ matrix
Bellman-Ford	Single-source	Yes	$O(VE)$
Floyd-Warshall	All-pairs	Yes	$O(V^3)$

Note

Negative cycles require special handling in shortest-path problems.

Warshall vs Floyd-Warshall

Feature	Warshall	Floyd-Warshall
Goal	Reachability	Minimum distance
Input	Boolean adjacency	Weighted adjacency
Output	Path matrix	Distance matrix
Operation	OR and AND	Min and addition
Time	$O(V^3)$	$O(V^3)$

Choosing the Right Algorithm

Problem	Algorithm
Shortest path in an unweighted graph	BFS
Single-source, nonnegative weights	Dijkstra
Single-source, negative weights	Bellman-Ford
All-pairs reachability	Warshall
All-pairs shortest distances	Floyd-Warshall
Minimum spanning tree	Prim / Kruskal

Exercise 1: Negative Edge

Question

Consider:

$$A \to B = 7$$

$$B \to C = -4$$

Find the path cost from A to C through B.

Solution

$$7 + (-4) = \boxed{3}$$

Exercise 2: Bellman-Ford

Question

A graph contains six vertices.
How many full relaxation passes are sufficient before the negative-cycle check?

Solution

$$V - 1 = 6 - 1$$

$$\boxed{5 \text{ passes}}$$

Exercise 3: Relaxation

Question

Given:

$$dist[u] = 5$$

$$w(u, v) = -3$$

$$dist[v] = 8$$

Find the updated distance.

Solution

$$5 + (-3) = 2 < 8$$

$$\boxed{dist[v] = 2}$$

Exercise 4: Warshall

Question

If:

$$P[A][B] = 1$$

and:

$$P[B][C] = 1$$

what can be concluded?

Solution

A can reach C through B.

$$\boxed{P[A][C] = 1}$$

Exercise 5: Floyd-Warshall

Question

Suppose:

$$D[A][C] = 12$$

$$D[A][B] = 4$$

$$D[B][C] = 5$$

Find the updated distance from A to C.

Solution

$$\min(12, 4 + 5) = 9$$

$$\boxed{D[A][C] = 9}$$

MCQ 1

Bellman-Ford can handle:

1. Only positive edges
2. Negative edge weights
3. Only unweighted graphs
4. Only trees

Correct Answer

Option 2.

MCQ 2

Bellman-Ford performs up to how many relaxation passes before cycle detection?

1. V
2. $V - 1$
3. $E - 1$
4. V^2

Correct Answer

Option 2: $V - 1$.

MCQ 3

Warshall's algorithm computes:

1. Minimum spanning tree
2. Transitive closure
3. Single-source shortest paths
4. Topological ordering

Correct Answer

Option 2.

MCQ 4

Floyd-Warshall computes:

1. All-pairs shortest paths
2. Minimum spanning tree
3. Graph traversal only
4. Connected components only

Correct Answer

Option 1.

MCQ 5

Warshall's algorithm uses:

1. OR and AND
2. Multiplication only
3. Division and subtraction
4. Sorting

Correct Answer

Option 1.

MCQ 6

The time complexity of Floyd-Warshall is:

1. $O(V)$
2. $O(V^2)$
3. $O(V^3)$
4. $O(E \log V)$

Correct Answer

Option 3.

MCQ 7

A negative diagonal entry after Floyd-Warshall indicates:

1. A disconnected graph
2. A negative-weight cycle
3. A minimum spanning tree
4. A zero-weight edge

Correct Answer

Option 2.

Complexity Summary

Algorithm	Time	Auxiliary Space
Bellman-Ford	$O(VE)$	$O(V)$
Warshall	$O(V^3)$	$O(1)$ in-place
Floyd-Warshall	$O(V^3)$	$O(1)$ in-place

Note

Warshall and Floyd-Warshall require $O(V^2)$ space for their working matrices.

Key Takeaways

- Bellman-Ford handles negative edge weights.
- Bellman-Ford uses repeated relaxation.
- A further scan detects reachable negative cycles.
- Warshall computes graph reachability.
- Warshall uses Boolean matrix operations.
- Floyd-Warshall computes all-pairs distances.
- Floyd-Warshall uses dynamic programming.
- Both matrix algorithms take $O(V^3)$ time.

Graph Algorithms: Final Overview

Algorithm	Main Purpose
BFS	Level-order traversal
DFS	Depth-oriented traversal
Union-Find	Dynamic connectivity
Kruskal	Minimum spanning tree
Prim	Minimum spanning tree
Dijkstra	Nonnegative single-source shortest paths
Bellman-Ford	Negative-weight single-source shortest paths
Warshall	Reachability
Floyd-Warshall	All-pairs shortest paths

Questions for Revision

1. Define a negative-weight cycle.
2. Explain Bellman-Ford's algorithm.
3. Describe edge relaxation.
4. Explain negative-cycle detection.
5. Compare Bellman-Ford and Dijkstra.
6. Define transitive closure.
7. Explain Warshall's recurrence.
8. Explain Floyd-Warshall's recurrence.
9. Perform matrix-based dry runs.
10. Compare the complexities of all shortest-path algorithms.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 13: Graphs.
2. Thomas H. Cormen et al., *Introduction to Algorithms*.
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*.

Student Feedback

Share Your Feedback



Thank you!